

the payment service have no thread connecting them, the engineer ends up running grep across a million log lines praying for the right one. The thread that holds these together is usually a trace ID stamped onto every log line emitted during that request, so one click on a slow span pulls up exactly the log entries it produced.

 **Note:** An open source project called OpenTelemetry is the standard most observability tools have agreed on for emitting and linking these signals, which means you do not end up stuck with the vendor.

Where to start if you are new to this

Nobody sets up all three on the same day. The realistic order is metrics, then logs, then traces, with some real time between each.

Metrics are cheap, fast to put in place, and pay off the soonest. A handful of dashboards covering the four golden signals catches most of what an on-call team needs to know in the first six months.

You will not yet have the answer to why. You will have the answer to when, which is the harder one to be without.

Logs are next, mostly because you already have some. Almost every system emits logs by default. The real work is not generating them -- it is cleaning them up. Make them structured (JSON, not free text). Drop the ones nobody is ever going to search. Get the storage bill under control before somebody in finance starts asking questions.

Traces come last, and only when your system really needs them. If your application is one program running on one server, a trace will show you nothing a log was not already showing you. Tracing earns its place when a single request starts crossing service boundaries (two services, three, ten), and not before. Setting up tracing for a monolith is mostly busy work.

Closing the loop

Logs, metrics, and traces are not three competing tools fighting for your attention. They are three different questions you can ask of the same running

system, and the real skill is knowing which question to ask first. Get that one call right and the rest of the investigation tends to unspool on its own.

Even with all three set up and linked to each other, observability still leans on someone in the room who knows where to look. The signals will show you everything. You can still miss it if nobody asks the right question. Tools do not replace judgment. They make judgment faster, which is not the same thing.

As systems keep splitting into more services running in more places (and that trend is not reversing), the gap between teams that read these signals fluently and those that only collect them is going to keep widening. The encouraging part is that the fluency is teachable. It usually starts by figuring out which question each signal was built to answer, and then trusting the signal to do its job. 

 **By: Raj Patel**

The author is a SaaS enthusiast and technical content creator.

■ ■ Continued from page...45

To sum it up, there are quite a few open source solutions to secure data while in storage or when being changed between locations. They can be used for free. One way is to create an open source stack using the most up-to-date security protocols for all network connections -- use WireGuard to create private tunnels for business communication, LUKS for full disk encryption on all company devices, GnuPG or Age to provide file-level encryption, a database encryption method to protect sensitive information contained within a database, and HashiCorp Vault or OpenBao to secure the keys that tie everything together.

By using these methods, an organisation can easily meet or exceed the level of protection expected from commercially available security

options. Encryption must be an organisational priority — the processes associated with certificate/key lifecycle management must be automated so that they are no longer dependent on the

memory of individuals. Organisations that invest in these practices will ensure their data is always available regardless of any other technical controls that may fail. 

References

- OpenSSL Software Foundation. (2024). OpenSSL: Cryptography and SSL/TLS Toolkit Documentation.
- Internet Security Research Group. (2024). Let's Encrypt: A Free, Automated, and Open Certificate Authority.
- Broz, M., et al. (2024). Cryptsetup and LUKS: Hard Disk Encryption for Linux Reference Documentation. GitLab.
- Donenfeld, J. A. (2017). WireGuard: Next Generation Kernel Network Tunnel. Proceedings of the Network and Distributed System Security Symposium (NDSS).
- Barker, E., & Roginsky, A. (2019). Transitioning the Use of Cryptographic Algorithms and Key Lengths (NIST Special Publication 800-131A Rev. 2). National Institute of Standards and Technology.

 **By: Sanjana Bhavsar**

The author is an AI/ML engineer and researcher.